PYTHON PROGRAMMING
ASSIGNMENT – CITY AIR-QUALITY MONITORING NETWORK

Name: Hemnath R

Reg No: 1920260116

Course Code: CSA0822

Github Link:https://github.com/hemnathr08-commits/python-programming-assignment_

PART A – DATA AND PROGRAM DESIGN

A1. Design the DataAcquisition

A2. Design the Cleaning and Normalisation

A3. Design the Analysis and Visualisation

A4. Plan Debugging and Profiling

A5. Design an AI Application for Citizen Complaints

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

B1. Implement andValidatetheCleaningPipeline

B2. Engineer Adversarial Data

B3. Loops vs Libraries vs SQL

B4. Final Engineering Decision

APPENDICES

Appendix A – Complete Python Program

Appendix B – Sample Input

Appendix C – Sample Output

Appendix D – Conclusion

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to design and implement a reliable Python data-processing system for a city air-quality monitoring network. The system receives daily CSV readings from monitoring stations and citizen complaints as text. The raw data may contain missing values, invalid negative values, duplicate rows, inconsistent formats and sensor faults.

The proposed workflow is:

ACQUIRE -> INSPECT -> CLEAN -> NORMALISE -> STORE -> QUERY -> VISUALISE -> PROFILE -> INTERPRET

The solution uses Python's csv, sqlite3, statistics, time, cProfile and logging modules. Matplotlib is used for charts when installed. The program also includes a rule-based sentiment scorer, complaint

PART A – DATA AND PROGRAM DESIGN

TASK A1 – DESIGN THE DATA ACQUISITION

A1.1 – Inspect the Source

The station CSV file is expected to contain fields such as:

station,date,hour,pm25,pm10,no2,temperature

Expected data-quality problems:

1. Missing values: PM2.5 may be blank or contain "NA".
Example: Adyar,2026-10-02,,35,18,29,31

2. Negative readings: a sensor may report an impossible negative concentration.
Example: Guindy,2026-10-02,-5,42,20,30

3. Implausibly high readings: a sensor fault may produce a value above the permitted PM2.5 validation limit of 999.
Example: Adyar,2026-10-03,1200,40,25,31

4. Duplicate rows: the same station, date and hour may be uploaded twice.

5. Inconsistent date formats: 2026-10-01, 01/10/2026 and 1-Oct-2026 may represent the same date.

6. Inconsistent station names: "Adyar", " adyar " and "ADYAR" should be treated as one station after normalisation.

7. Invalid numeric values: a reading may contain text such as "bad" instead of a number.

8. Wrong delimiter: a file may use semicolons instead of commas.

9. Missing columns: a file may not contain the required PM2.5 column.

10. Encoding problems: a file may be saved in a different text encoding.

11. Fahrenheit temperature: some stations may report Fahrenheit while others report Celsius. A unit field or documented source rule is required before conversion; the system must not guess silently.

A1.2 – Design the Storage

A relational SQLite database is proposed.

Table 1: stations

Column	Type	Description
station_id	INTEGER	Primary key
station_name	TEXT	Unique, normalised station name

Table 2: readings

Column	Type	Description
reading_id	INTEGER	Primary key
station_id	INTEGER	Foreign key to stations
reading_date	TEXT	ISO date in YYYY-MM-DD format
hour	INTEGER	Hour from 0 to 23; nullable for daily records
pm25	REAL	Validated non-negative PM2.5 value
pm10	REAL	Optional PM10 value
no2	REAL	Optional NO2 value
temperature_c	REAL	Optional temperature in Celsius

A unique constraint on (station_id, reading_date, hour) prevents duplicate station readings for the same time. For daily data without an hour, a separate uniqueness policy should be used. Indexes on station_id and reading_date support station and date filtering.

CSV-to-SQL mapping:

CSV station -> stations.station_name

CSV date -> readings.reading_date

CSV hour -> readings.hour

CSV pm25 -> readings.pm25

CSV pm10 -> readings.pm10

CSV no2 -> readings.no2

CSV temperature -> readings.temperature_c, after verified conversion

The station name is stored once in the stations table. Readings refer to the station through station_id, avoiding repeated station-name storage.

A1.3 – CSV or SQL

Question 1: What is the average PM2.5 at Adyar in October?

CSV: read all rows, filter Adyar and October, then calculate the mean.

SQL: SELECT AVG(pm25) FROM readings JOIN stations USING(station_id)
WHERE station_name = ? AND reading_date >= ? AND reading_date < ?;

Question 2: Which station has the highest daily mean?
CSV: group all rows by station and date in Python.
SQL: use GROUP BY station_id, reading_date and ORDER BY AVG(pm25) DESC.

Question 3: How many valid readings were recorded at Guindy?
CSV: scan every row and count matching valid rows.
SQL: SELECT COUNT(*) FROM readings JOIN stations USING(station_id)
WHERE station_name = ?;

Complexity comparison:

For n CSV rows, each full scan is O(n). Repeating a scan for q questions
can cost O(qn). SQL also scans O(n) without a useful index, but indexes
can reduce selective lookups to approximately O(log n + k), where k is
the number of matching records. SQL adds disk storage and index overhead,
but it avoids loading the entire dataset into memory for every query.

Alternative concept and complexity analysis:
If data were kept only in CSV files and loaded into a pandas DataFrame
for every question, each reload would require reading O(n) data and
memory of O(n). Keeping the data in SQLite allows persistent storage and
indexed queries. For a small one-time file, CSV may be simpler; for
repeated queries, many stations and daily automated reporting, SQLite is
more suitable.

Evidence expected:
A documented source schema, examples of data-quality problems, a justified
database design and a reasoned CSV-versus-SQL comparison.

TASK A2 – DESIGN THE CLEANING AND NORMALISATION

A2.1 – Cleaning Rules

Problem Rule
Missing PM2.5 For a time series, fill from previous valid
reading only when the gap is short and the
carry-forward rule is approved. Otherwise use
the station median or mark missing. Do not
silently invent values.

"NA" or blank	Treat as missing, not as zero.
Negative PM2.5	Reject as physically invalid.
PM2.5 above 999	Reject as an implausible sensor-fault value.
Duplicate row	Keep one record using the unique station, date and hour key; log the duplicate.
Invalid date	Reject and record the reason.
Invalid station name	Trim spaces and use consistent title case.
Invalid numeric text	Reject or mark missing according to the field.
Fahrenheit temperature	Convert using C = (F - 32) * 5 / 9 only when the source confirms Fahrenheit.

Justification:
Dropping invalid negative or extreme sensor-fault readings protects the mean from impossible values. Missing values should not automatically be replaced by zero because zero would lower the average. Forward fill is appropriate only for short gaps in time-series data where the previous reading remains representative. Median filling is less sensitive to extreme observations, but a filled value must be marked as imputed.

A2.2 – Normalisation

Min-max normalisation:
x' = (x - minimum) / (maximum - minimum)

It maps PM2.5 values to the range 0 to 1. It is useful when charting variables with different scales. It is sensitive to extreme values.

Z-score standardisation:
z = (x - mean) / standard deviation

It expresses a reading as the number of standard deviations from the mean. It is useful for comparing unusual readings and statistical analysis. It is not limited to the range 0 to 1.

If maximum equals minimum, min-max normalisation returns 0 for every value because the range is zero. If standard deviation is zero, z-scores are defined as 0 for a constant dataset to avoid division by zero.

A2.3 – Formatting

- Dates: YYYY-MM-DD (ISO 8601).
- Station names: trimmed, title case and consistent spelling.
- PM2.5: numeric floating-point values in the source's documented unit.

• Hour: integer from 0 through 23.

• Missing optional values: SQL NULL, not the text "NA".

• Temperature: Celsius only after a verified unit conversion.

Alternative concept and complexity analysis:
Dropping all missing readings is simple and takes O(n) time, but it may bias daily averages if missingness is systematic. Forward fill also takes O(n) time in one pass and uses O(1) extra state per station when rows are ordered by station and time. Median filling generally requires finding or sorting values; sorting takes O(n log n) time and O(n) storage in a straightforward implementation. The chosen strategy should preserve missingness information and document imputation.

Evidence expected:
Justified cleaning rules, normalisation equations and standard formats.

TASK A3 – DESIGN THE ANALYSIS AND VISUALISATION

A3.1 – Choose the Charts

Board question	Chart and reason
Trend of PM2.5 over a month	Line chart: dates are ordered and trends are easy to observe.
Compare stations	Box plot: compares median, spread and possible outliers per station.
Temperature vs PM2.5 between two numeric variables. Hours with worst air	Scatter plot: shows relationship
hour and identifies high periods.	Bar chart: compares mean PM2.5 by
Distribution of PM2.5 concentration ranges.	Histogram: shows frequency across

Every chart must have a descriptive title, labelled axes and units.
Charts should be generated from cleaned records only.

A3.2 – Design the Summary Table

Daily summary table:

Column	Meaning
station_name	Monitoring station
reading_date	Date

minimum_pm25 Lowest valid PM2.5 reading

maximum_pm25 Highest valid PM2.5 reading

mean_pm25 Average PM2.5

valid_readings Number of valid observations

SQL production:

```
SELECT station_name, reading_date, MIN(pm25), MAX(pm25),
AVG(pm25), COUNT(*)
FROM readings JOIN stations USING(station_id)
GROUP BY station_name, reading_date;
```

The SQL GROUP BY performs the aggregation in the database. It avoids transferring all raw readings to Python and is appropriate for daily automated reports.

Alternative concept and complexity analysis:
Computing daily summaries in Python loops requires O(n) time and dictionary storage proportional to the number of station-day groups g. SQL aggregation also processes the n rows, typically O(n) without sorting/index effects, and returns only g summary rows. The database approach reduces data transfer and keeps the summary logic close to the stored data.

Evidence expected:
Justified chart choices, a summary-table design and chart interpretation.

TASK A4 – PLAN DEBUGGING AND PROFILING

A4.1 – Debugging Plan

1. print(): inspect a small sample of raw rows, cleaned rows and rejected rows during initial development. Remove or disable verbose prints in production.
2. assert: verify invariants such as PM2.5 >= 0, PM2.5 <= 999 and date format validity during tests.
3. logging: record file errors, missing columns, rejected rows, duplicate records and database failures with timestamps and severity levels.
4. pdb: set a breakpoint near the cleaning rule and inspect variables, the current row and the reason for rejection.
5. IDE debugger: use breakpoints, step-over, step-into and watch expressions to locate incorrect values.

A4.2 – Profiling Plan

• time.perf_counter(): measure elapsed wall-clock time around a complete operation. Use repeated runs and report the best or median result.
• timeit: compare small functions such as CSV parsing and normalisation under repeatable conditions.
• cProfile: identify which functions consume the most cumulative time.
• Profile separate stages: CSV reading, cleaning, SQLite insertion, SQL aggregation, chart generation and complaint processing.
• Record row count, elapsed time and memory where possible.

Alternative concept and complexity analysis:
Using print statements for every row can generate thousands of output operations and distort timing. Logging supports configurable levels and can be disabled or reduced in production. A debugger is best for investigating a specific failure, while profiling is used to locate performance bottlenecks. These tools serve different purposes.

Evidence expected:
A clear debugging and profiling plan suited to the program.

TASK A5 – DESIGN AN AI APPLICATION FOR CITIZEN COMPLAINTS

A5.1 – Sentiment and Summary

A simple word-list sentiment scorer is designed using positive and negative vocabulary. A preceding negator such as "not", "never" or "no" reverses the next recognised sentiment word.

Example:
"The air is clean and helpful." -> positive
"Smoke is bad and the air is dirty." -> negative
"The service is not good." -> negative
"Not bad, service improved." -> positive under the simple one-word negation rule.

The extractive summariser tokenises complaint sentences, counts important words and selects the sentence with the highest frequency-based score.
It does not create new facts; it selects existing complaint sentences.

A5.2 – Basic Chatbot

The rule-based chatbot recognises keywords:

• "PM2.5" or "pollution": query latest available PM2.5 information.

• A station name such as "Adyar": filter by that station.

• "average": return the station average.

• "highest": return the station with the highest mean.

Limitations:

A word-list classifier cannot reliably understand sarcasm, complex negation, context, spelling mistakes or mixed opinions. A keyword chatbot cannot answer questions outside its predefined patterns. A trained classifier may improve context recognition but needs labelled training data, more memory and inference time, and can still make errors.

Alternative concept and complexity analysis:

A word-list scorer processes t tokens in O(t) time and uses O(v) storage for a vocabulary of size v. A trained model has model-loading memory overhead and inference cost depending on model size and input length. The simple rule-based solution is fast, transparent and suitable as a baseline for a small system; a trained model should be considered only after collecting representative labelled complaints and measuring accuracy.

Evidence expected:

Clear designs for the sentiment scorer, summariser and chatbot, with limitations.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

The implementation follows:

IMPLEMENT -> INSTRUMENT -> STRESS -> MEASURE -> COMPARE -> INTERPRET

The program uses the Python standard library. Matplotlib is optional for visualisation. A sample CSV is supplied in Appendix B.

TASK B1 – IMPLEMENT AND VALIDATE THE CLEANING PIPELINE

B1.1 – Validate the Implementation

The cleaning function reads CSV using csv.DictReader. It:

• checks that station, date and pm25 columns exist;

• skips blank, "NA", negative and above-999 PM2.5 values;
• validates dates and converts them to ISO format;
• normalises station names;
• removes duplicate station-date-hour keys;
• records rejected rows and reasons.

A test file should contain each listed problem. The supplied sample input contains valid, missing, negative, extreme and duplicate examples.

B1.2 – Compare with Your Design

The implementation follows the Part A design. The main difference is that the demonstration code rejects missing PM2.5 values rather than automatically filling them. This is safer when no approved imputation policy or time-series context is supplied. In production, a documented short-gap forward-fill or station-median strategy can be added with an imputation flag.

B1.3 – Store and Query

Clean records are inserted into SQLite using executemany() and parameter placeholders. The program runs:
1. Average PM2.5 for Adyar.
2. Highest station-day mean.
3. Valid reading count for Guindy.

Parameterized queries prevent SQL injection because user input is treated as data, not executable SQL.

B1.4 – Visualise

The program generates:
• a line chart of daily mean PM2.5;
• a bar chart comparing station means;
• a scatter plot when temperature values are available.

Interpretation must be based on the actual generated dataset. With the small demonstration data, the chart illustrates the process and is not a statistically representative city-wide conclusion.

TASK B2 – ENGINEER ADVERSARIAL DATA

Test cases:

1. Missing column: remove pm25. The program should report a missing required column rather than silently treating the file as valid.
2. Wrong delimiter: provide a semicolon-separated CSV. The program should detect or report that required columns are unavailable.
3. Different text encoding: read UTF-8 input and report decoding errors. For real source files, use the encoding specified by the provider.
4. Empty file: report that there are no records to process.
5. SQL injection attempt: search for station name Adyar' OR '1'='1. The parameterised query should return no matching station, not every database row.
6. Sentiment sarcasm: "Great, another day breathing toxic smoke!" may be labelled positive because of "great". This demonstrates a limitation.
7. Double negative: "not bad" is handled by one negator; more complex forms such as "not never good" are not reliably interpreted.
8. Mixed opinion: "The air is clean but traffic smoke is bad" may receive a neutral or mixed score depending on word counts.

The injection-safe query uses:

```
SELECT AVG(pm25) FROM readings
JOIN stations USING(station_id)
WHERE station_name = ?;
```

Unsafe string concatenation can allow an input such as Adyar' OR '1'='1 to change the SQL condition. A placeholder keeps the input as a literal value.

Alternative concept and complexity analysis:
Building a SQL string for every row is error-prone and exposes injection risk. executemany() with placeholders reuses a parameterised statement and reduces repeated Python-to-SQL calls. The database still validates and stores each row, but batching improves practical throughput.

Evidence expected:
Deliberately constructed adversarial inputs, observed behaviour and reasoned explanation.

TASK B3 – LOOPS VS LIBRARIES VS SQL

B3.1 – Correctness on Small Data

Daily means are computed in three ways:

1. Plain Python loops using a dictionary of sum and count.
2. pandas groupby, when pandas is installed.
3. SQL GROUP BY in SQLite.

All approaches should produce the same daily mean within a small floating-point tolerance. The supplied program includes loop and SQL implementations. The pandas comparison is optional if pandas is not installed.

B3.2 – Increase the Data Size

Generate synthetic data for:

• 1 station;
• 12 stations;
• 120 stations;
• 8,760 hourly readings per station for a full non-leap year.

The total number of records is approximately 8,760, 105,120 and 1,051,200 respectively. Measure the loop and SQL aggregation times. Use the same generated data for both approaches.

B3.3 – Profile

Run cProfile on the plain-Python implementation. The report should identify functions with the highest cumulative time. Likely work includes CSV parsing, dictionary aggregation and database insertion. Actual results depend on hardware, Python version and storage.

B3.4 – Engineering Trade-Off

For a quick one-time analysis of a small CSV file, a Python loop is simple and avoids database setup. For a daily automated report and repeated queries, SQLite is preferred because it provides persistent storage, SQL aggregation and indexes. pandas is convenient for exploration and vectorised operations but can require substantial memory for very large datasets.

Alternative concept and complexity analysis:
A running sum and count per station can compute daily means in one pass. For n readings and s station-day groups, time is O(n) and storage is O(s), rather than retaining all n readings in memory. SQL GROUP BY also processes the records and can use indexes and database query planning.

TASK B4 – FINAL ENGINEERING DECISION

B4.1 – Implement a Practical Strategy

The word-list sentiment scorer and extractive summariser are implemented. They are tested on a small set of complaints. Agreement is measured by comparing predicted labels with manually assigned expected labels.
For a proper evaluation, use at least 50 labelled complaints and report correct predictions / total complaints as accuracy.

B4.2 – Check the Database

An index is created on (station_id, reading_date). Compare query time before and after the index using the same query and dataset. Indexes improve selective lookups but require disk space and add insertion overhead.

B4.3 – Production Decision

Recommended production design:

• Store validated readings in SQLite for a small-to-medium deployment.
For a larger deployment, use a managed relational database with backups, access control and scheduled ingestion.
• Preserve raw input files for audit and reprocessing.
• Store rejected rows and reasons separately.
• Keep imputed readings distinguishable from measured readings.
• Use SQL aggregation for daily and monthly reports.
• Use parameterised queries for all user input.
• Use a word-list sentiment baseline initially; evaluate a trained classifier only after collecting labelled local-language complaints.
• Monitor processing time, invalid-row rate, missingness and query time.

At 120 stations and hourly readings, the annual volume is about 1.05 million records. SQLite can handle this scale for a single-writer, moderate-query workload with suitable indexes. If many concurrent writers, distributed ingestion or high availability are required, a server database is a better production choice.

Evidence expected:
Working AI functions with an agreement measure, index timings and an evidence-based production decision.

APPENDIX A – COMPLETE PYTHON PROGRAM

The complete executable source is provided separately as:

city_air_quality.py

It includes:

• CSV inspection and cleaning;

• normalisation;

• SQLite creation, bulk insertion and parameterised queries;

• daily summary generation;

• chart generation;

• loop and SQL aggregation;

• synthetic scale-data generation and timing;

• cProfile entry point;

• sentiment scoring and complaint summarisation;

• a keyword chatbot;

• adversarial input tests.

Run:

python city_air_quality.py

Optional charts:

Install matplotlib using: python -m pip install matplotlib

Optional pandas comparison:

Install pandas using: python -m pip install pandas

Run profiling:

python -m cProfile -s cumulative city_air_quality.py

APPENDIX B – SAMPLE INPUT

File: sample_air_quality.csv

station,date,hour,pm25,temperature_c

Adyar,2026-10-01,8,48,30 Adyar,2026-

10-01,9,52,31 Adyar,2026-10-

02,8,NA,30 Adyar,2026-10-02,9,55,31

Guindy,2026-10-01,8,95,29

Guindy,2026-10-01,9,95,30

Guindy,2026-10-02,8,-5,29

Guindy,2026-10-02,9,121,30

Adyar,2026-10-01,8,48,30

Adyar,2026-10-03,8,1200,32

Guindy,2026-10-03,8,121,31

Expected cleaning:

• Keep 7 valid unique rows.

• Reject the missing PM2.5 row.

• Reject the negative PM2.5 row.

• Reject the value 1200 because it is above 999.

• Reject the duplicate Adyar 2026-10-01 hour 8 row.

• Keep the valid Guindy and Adyar measurements.

APPENDIX C – SAMPLE OUTPUT

A representative output for the sample file is shown below. Execution times and chart display may vary by computer.

CLEANING SUMMARY

Valid rows: 7

Rejected rows: 4

REJECTION REASONS

missing/NA PM2.5: 1

negative PM2.5: 1

PM2.5 above 999: 1

duplicate record: 1

DAILY SUMMARY

Adyar | 2026-10-01 | min=48.00 | max=52.00 | mean=50.00 | count=2 Adyar | 2026-10-02 | min=55.00 | max=55.00 | mean=55.00 | count=1 Guindy | 2026-10-01 | min=95.00 | max=95.00 | mean=95.00 | count=2 Guindy | 2026-10-02 | min=121.00 | max=121.00 | mean=121.00 | count=1 Guindy | 2026-10-03 | min=121.00 | max=121.00 | mean=121.00 | count=1

QUERY RESULTS

Adyar average PM2.5: 51.67

Highest station-day mean: Guindy, 2026-10-02, 121.00 (tie with 2026-10-03)

Guindy valid reading count: 4

SENTIMENT EXAMPLES

positive | Roads are clean and the bus was fast.

negative | Smoke from the factory, air is bad.

negative | The staff were not helpful.

positive | Not bad, service improved.

SECURITY TEST

Injection-like station name returns no matching row.

The parameterised query does not return all records.

NOTE:

The report's output values above are calculated from the exact sample input listed in Appendix B. Timing results are machine-dependent. If the program is run with a different input file, database and chart values will change accordingly.

APPENDIX D – CONCLUSION

This assignment demonstrates a complete data workflow for a city air-quality monitoring network. The source data is inspected, invalid readings are rejected, station names and dates are standardised, and validated measurements are stored in SQLite. SQL queries generate station averages, counts and daily summaries. Charts help communicate trends and differences between stations.

The experimental section compares Python loops, SQL aggregation and optional pandas groupby. Profiling and timing identify bottlenecks as the dataset grows. Parameterised SQL protects against injection attempts. The word-list sentiment scorer, extractive summariser and keyword chatbot provide a simple baseline for citizen complaints, while their limitations are documented.

The recommended design preserves raw files, records rejected rows, stores clean data in a structured database, uses indexes for repeated queries and evaluates AI components against manually labelled examples. The system therefore focuses not only on producing charts, but also on data quality, correctness, performance, security and maintainability.